University of Mascara
Department of Computer sciences
Second Year Engineer

Algorithmic and Complexity

# Problem Set 4

Tree

***Problem 1*** Implement function to resolve the following problems in a binary tree:

1) Sum of all values, 2) Number of occurrences of a value, 3)Finding the max/min value, 4) count: Full Nodes (have both a left and a right child), Half Nodes (with one child) and Right Leaves (leaf node that is the right child of its parent)

***Problem 2*** write functions to:

- Find the smallest value in BST
- Check if a binary tree satisfies the BST properties.
- Find the lowest common ancestor (LCA) of two given nodes (If root->info is between n1 and n2, root is the LCA . If root->info is greater than both n1 and n2, search in the left subtree. If root->info is less than both n1 and n2, search in the right subtree)
- Store the elements of the BST in a sorted array

***Problem 3*** Implement functions to check if a binary tree is:

1) Full, 2) Complete, 3) BST, 4) Balanced, 5) Symmetric (its left and right subtrees are mirror images of each other)

***Problem 4*** In general tree, Write a function to

1) Create a new node for the general tree, 2) Add a child node to a given parent node. If the parent already has children, add the new node as the rightmost sibling.
3) Count the total number of nodes, 4) count the number of leaf nodes, 5) Calculate the height of the general tree, 6) Find the order of a tree (maximum number of children)

***Problem 5***

- Perform a level order traversal (BFS) of a binary tree
- What does the following function do ? Estimate its complexity

```
NPtr find(NPtr root, int n1, int n2) {
    if (root == NULL)  return NULL;
    if (root->info == n1 || root->info == n2)  return root;
    NPtr leftF = find(root->left, n1, n2);
    NPtr rightF = find(root->right, n1, n2);
    if (leftF != NULL && rightF != NULL)   return root;
    return (leftF != NULL) ? leftLF : rightF;
}
```